

Phase 1, Part 3: Develop the Search Tool (Serper.dev Integration)

With the financial data tool ready, let's build the second crucial component: the Search Tool. This tool will use Serper.dev to perform web searches, allowing your agent to gather real-time news, articles, and other relevant information for its financial research.

Step 1: Install langchain-community

If you haven't already, ensure your virtual environment is activated and install the `langchain-community` library. This library provides the `GoogleSerperAPIWrapper` which simplifies interaction with Serper.dev.

Bash

```
source venv/bin/activate # On Windows, use `venv\Scripts\activate`  
pip install langchain-community
```

Step 2: Create the Search Tool in tools.py

Open your `tools.py` file and add the following Python code. This code defines a function `perform_web_search` that takes a query as input and uses `GoogleSerperAPIWrapper` to get search results.

Python

```
# tools.py (append to the existing content)  
  
import os  
from langchain_community.utilities import GoogleSerperAPIWrapper  
  
# Ensure environment variables are loaded if running tools.py directly for test.  
# from dotenv import load_dotenv  
# load_dotenv()  
  
def perform_web_search(query: str) -> str:  
    """  
    Performs a web search using Serper.dev and returns the top results.  
    """  
    try:  
        # Ensure SERPER_API_KEY is loaded from .env  
        serper_api_key = os.getenv("SERPER_API_KEY")  
        if not serper_api_key:
```

```

        return "Error: SERPER_API_KEY not found in environment variables."

    search = GoogleSerperAPIWrapper(serper_api_key=serper_api_key)
    results = search.results(query)

    # Format the results for better readability
    formatted_results = []
    if 'organic' in results:
        for i, res in enumerate(results['organic'][:5]): # Get top 5 organic results
            title = res.get('title', 'No Title')
            link = res.get('link', 'No Link')
            snippet = res.get('snippet', 'No Snippet')
            formatted_results.append(f"Result {i+1}: \nTitle: {title} \nLink: {link} \nSnippet: {snippet}")

    if not formatted_results:
        return f"No search results found for '{query}'."

    return "\n".join(formatted_results)

except Exception as e:
    return f"Error performing web search for '{query}': {e}"

# Example usage (for testing purposes, can be removed later)
if __name__ == "__main__":
    # Add this block to the end of your tools.py for testing
    # Make sure to uncomment load_dotenv() if running tools.py directly
    from dotenv import load_dotenv
    load_dotenv()

    print("\n--- Testing Financial Data Tool ---")
    apple_info = get_stock_info("AAPL")
    print(apple_info)

    print("\n--- Testing Web Search Tool ---")
    news_query = "latest news on Apple Inc."
    apple_news = perform_web_search(news_query)
    print(apple_news)

    invalid_search = perform_web_search("asdfghjklqwertyuiopzxcvbnm_nonexistent")
    print(invalid_search)

```

Step 3: Test Your Search Tool

To ensure your `perform_web_search` function works as expected, run `tools.py` directly from your terminal:

Bash

```
python tools.py
```

You should see output containing financial information for Apple (from the previous tool) and then the top 5 search results for "latest news on Apple Inc." (from your new search tool). If you encounter an error related to `SERPER_API_KEY`, ensure it's correctly set in your `.env` file and `load_dotenv()` is uncommented in the `if __name__ == "__main__":` block of `tools.py` for direct testing.

This output confirms that your Serper.dev integration is working, and your `perform_web_search` function can successfully retrieve and format web search results. Both the financial data tool and the search tool are now ready to be integrated into your LangChain agent in the next phase.